Appendix C:

Declaration on Use of GAI (Generative Artificial Intelligence) Assistance in relation to Assignment/Project (to be submitted individually even for group projects)

I CHIN ZHI KHANG (student name), ZCHIN012@e.ntu.edu.sg (NTU email) honestly and sincerely make the following declaration in relation to the following course submission:

1. Name of course: Object-Oriented Design and Programming

2. Course Code: SC2002

3. Instructor: Dr Li Fang

4. Title of Assignment/Project Submission: Turn-Based Combat Assignment

In relation to the foregoing I hereby declare that, fully and properly in accordance with the Assignment/Project Instructions I have (check where appropriate):

i. Used GAI as permitted to assist in generating key ideas only. ☑

ii. Used GAI as permitted to assist in generating a first text only. ☐

And/or

iii. Used GAI to refine syntax and grammar for correct language submission only ☑

Or

iv. As it is not permitted: Not used GAI assistance in any way in the development or generation of this assignment or project. ☐

I also declare that I have :

a. Fully and honestly submitted the digital paper trail required under the assignment/project instructions; and that

b. Wherever GAI assistance has been employed in the submission in word or paraphrase or inclusion of a significant idea or fact suggested by the GAI assistant, I have acknowledged this by a footnote; and that,

c. Apart from the foregoing notices, the submission is wholly my own work.

19/4/2026

Student Name & Signature Date

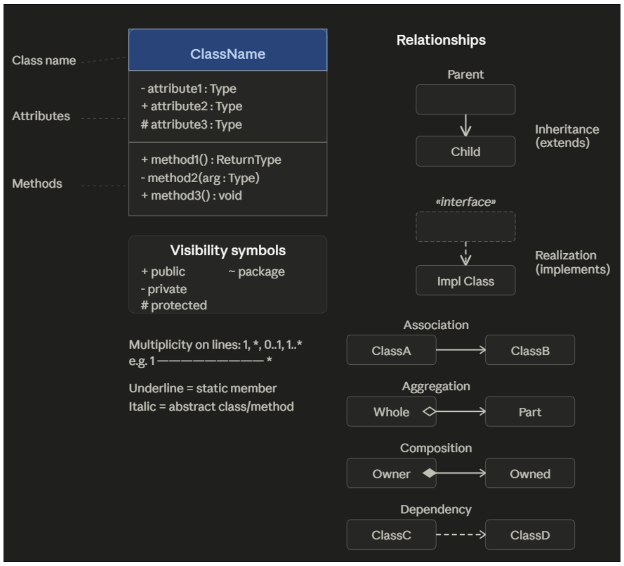
(Optional) Sample Reflection Questions:

In 150 words or less, briefly state up to 5 key ideas that you formulated that the GAI tools alone could not have generated.

Prompt: <u>i need to fill up UML class diagram based on my classes, what is the format?</u>

Here's the standard UML class diagram format:

Here's the standard UML class diagram format, along with a visual reference:



Here's a quick summary of the key parts:

Class box — three horizontal sections stacked:

1. Class name (bold, centered; italicized for abstract classes)
2. Attributes — visibility name : Type
3. Methods — visibility name(params) : ReturnType

Visibility prefixes — + public, - private, # protected, ~ package

Relationships:

- **Inheritance** — solid line + open hollow triangle (child → parent)
- **Realization** — dashed line + open hollow triangle (class → interface)
- **Association** — solid line + open arrow (has-a, general link)
- **Aggregation** — solid line + open diamond (weak ownership, "whole has parts")
- **Composition** — solid line + filled diamond (strong ownership, parts can't exist alone)
- **Dependency** — dashed line + open arrow (uses temporarily)

Multiplicity goes on the ends of lines: 1, *, 0..1, 1..*, etc.